

# **FIT2004 - Algorithms and Data Structures**

## Lecture 1 - Introduction

---

Rafael Dowsley

# Agenda

- ① Unit Information
- ② Divide-and-Conquer
- ③ Complexity Analysis
- ④ Solving Recurrence Relations

## Unit Information

---

# What is this unit about?

- Solving problems with computers - **efficiently**.
- Developing your algorithm toolbox.
- Training your problem solving skills.
- **The unit is not really about programming:**
  - ▶ The subject is really language agnostic.
  - ▶ Algorithms in this unit will be described in English, pseudocode, procedural set of instructions or Python.

# Is this unit important?

- Algorithms and Data Structures is a key unit in computer science degrees around the world.
- The subject is very important for careers in the area:
  - ▶ Companies actively hunt for people good at algorithms and data structures.
  - ▶ Many job interview questions are based on algorithms from this unit.
  - ▶ Many applied class questions are in fact very similar to questions you could be asked in job interviews.
  - ▶ **You are the future of CS and will make great contributions to field.**  
**What you learn in this unit will greatly help you throughout your career.**

# Overview of the contents

- Explore some of the most important algorithm design paradigms:
  - ▶ Divide-and-Conquer
  - ▶ Greedy algorithms
  - ▶ Dynamic Programming
  - ▶ Network Flow
- Analysis of algorithms.
- Learn important data structures for implementing algorithms efficiently.
- Algorithms for solving important computational problems.

## Additional references

- This subject has huge value for your professional development into careers in computer science. Some books you might want to refer to (from time-to-time, even beyond this unit) include:
  - ▶ CLRS: Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. *Introduction to Algorithms*.
  - ▶ KT: Jon Kleinberg, Éva Tardos. *Algorithm Design*.
  - ▶ Rou: Tim Roughgarden. *Algorithms Illuminated*.
  - ▶ Knu: Donald Knuth. *The Art of Computer Programming*. More advanced, pretty expensive; but iconic CS book!
- No required textbooks. Beware that all of those textbooks contain both less and (far) more than is required for the unit. That is why it is important to not rely solely on these books (i.e., you should watch the seminars, attend applied classes and read the course notes).

# Your first algorithm

- What was the first algorithm you learned?
  - ▶ As an CS student you already learned some algorithms in the university: binary search, sorting algorithms, etc.
  - ▶ But algorithms predate computers by millennia. Even the word 'algorithm' is derived from the name of a 9th century Persian mathematician.
  - ▶ In fact, you learned in school an algorithm that is more than 2000 years old: Euclid's algorithm for computing the greatest common divisor.
  - ▶ Even in your first school years you already learned the “grade school” multiplication algorithm.







# Divide-and-Conquer

---

# Divide-and-Conquer paradigm

- The Divide-and-Conquer algorithm design paradigm works in 3 steps:
  1. Divide the problem into smaller subproblems.
  2. Conquer (i.e., solve) the smaller subproblems.
  3. Combine the solutions of the smaller subproblems to obtain the solution of the bigger problem.
- Analysing the time complexity of a divide-and-conquer algorithm normally involves solving a recurrence relation.
- Normally a polynomial time solution to the problem is already known, and the divide-and-conquer strategy is used to reduce the time complexity to a lower polynomial.

# First improvement idea

- **Problem:** multiply two  $n$ -digits numbers  $a$  and  $b$  in sub-quadratic time given addition and multiplication of single digit numbers as the basic operations.

$$\begin{array}{r} a \\ \times \\ b \end{array}$$

- **Improvement idea:** split the numbers between the  $n/2$  most significant digits and the  $n/2$  least significant digits; and do recursive calls with them.
- From math we know that:

$$\begin{aligned} a \cdot b &= (a_M \cdot 10^{n/2} + a_L)(b_M \cdot 10^{n/2} + b_L) \\ &= a_M \cdot b_M \cdot 10^n + a_M \cdot b_L \cdot 10^{n/2} + a_L \cdot b_M \cdot 10^{n/2} + a_L \cdot b_L \end{aligned}$$

## Are we making progress?

$$a \cdot b = a_M \cdot b_M \cdot 10^n + a_M \cdot b_L \cdot 10^{n/2} + a_L \cdot b_M \cdot 10^{n/2} + a_L \cdot b_L$$

- Reduce 1 instance of the problem of size  $n$  to 4 instances of size  $n/2$ :

|          |          |          |          |
|----------|----------|----------|----------|
| $a_M$    | $a_M$    | $a_L$    | $a_L$    |
| $\times$ | $\times$ | $\times$ | $\times$ |
| $b_M$    | $b_L$    | $b_M$    | $b_L$    |

- Are we making progress?
- Not really! Intuition: 4 instances that will each take about  $\frac{1}{4}$  of the time that was necessary to solve the original problem, so the overall time stays in the same order. You can later check that by solving the recurrence  $T(n) = 4 \cdot T(n/2) + c \cdot n$ .
- If we want to follow this approach to improve the efficiency, we should use at most 3 recursive calls.

# Are improvements possible?

**Conjecture:** Sub-quadratic multiplication is impossible!



Andrey Kolmogorov, one of the greatest mathematicians of the 20th century.

**That is wrong!**



Anatoly Karatsuba, then a 23 y/o student, within one week from hearing that in his seminar.

# Karatsuba's algorithm

$$a \cdot b = a_M \cdot b_M \cdot 10^n + (a_M \cdot b_L + a_L \cdot b_M) \cdot 10^{n/2} + a_L \cdot b_L$$

- Do only 3 recursive calls to compute:

|          |          |             |
|----------|----------|-------------|
| $a_M$    | $a_L$    | $a_M + a_L$ |
| $\times$ | $\times$ | $\times$    |
| $b_M$    | $b_L$    | $b_M + b_L$ |
| $= (1)$  | $= (2)$  | $= (3)$     |

- Given the results of (1), (2) and (3), if we can trivially obtain  $(a_M \cdot b_L + a_L \cdot b_M)$  then we are done computing  $a \cdot b$  with only 3 recursive calls.



# The trick

- **Problem:** Obtain  $c = (a_M \cdot b_L + a_L \cdot b_M)$  without further recursive calls when given:

|          |          |             |
|----------|----------|-------------|
| $a_M$    | $a_L$    | $a_M + a_L$ |
| $\times$ | $\times$ | $\times$    |
| $b_M$    | $b_L$    | $b_M + b_L$ |
| $= (1)$  | $= (2)$  | $= (3)$     |

- Note that

$$\begin{aligned}(a_M + a_L) \cdot (b_M + b_L) &= a_M \cdot b_M + a_M \cdot b_L + a_L \cdot b_M + a_L \cdot b_L \\ &= c + a_M \cdot b_M + a_L \cdot b_L\end{aligned}$$

- **Solution:** We obtain  $c$  by computing  $(3) - (1) - (2)$ .
- Where does this trick come from?

# Where does this trick come from?

- This trick traces back to Gauss' method for multiplying complex numbers using 3 multiplications of real numbers instead of 4.

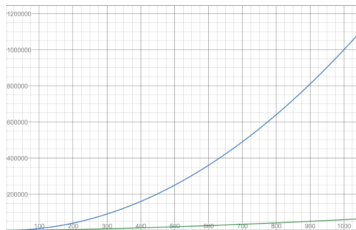


Johann Carl Friedrich Gauss

- **Adapting previous ideas to solve your new problem can be very useful!**
- *"It is not knowledge, but the act of learning, not the possession of but the act of getting there, which grants the greatest enjoyment."*
  - Johann Carl Friedrich Gauss

## Is that a big improvement?

- The time complexity of Karatsuba's algorithm is  $O(n^{1.59})$ . To verify that, just solve the recurrence  $T(n) = 3 \cdot T(n/2) + c \cdot n$  and use the fact that  $\log_2 3 < 1.59$ .
- Don't underestimate the difference between  $n^2$  and  $n^{1.59}$ !



- This is the algorithm that Python uses for multiplying large numbers.

# Merge Sort

- $O(n \log n)$  sorting algorithm presented by John von Neumann.
- One of the first explicit uses of the Divide-and-Conquer paradigm.
- Python uses as standard Timsort (a hybrid stable sorting algorithm derived from Merge Sort and Insertion Sort).

---

Merge sort

---

```
1: function MERGE_SORT(array[lo..hi])  
2:   if hi > lo then  
3:     mid =  $\lfloor (lo + hi) / 2 \rfloor$   
4:     MERGE_SORT(array[lo..mid])  
5:     MERGE_SORT(array[mid + 1..hi])  
6:     array[lo..hi] = MERGE(array[lo..mid], array[mid + 1..hi])
```

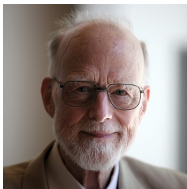
---



John von Neumann, one of the greatest mathematicians of the 20th century.

# Quick Sort

- Sorting algorithm created by Tony Hoare that uses the Divide-and-Conquer paradigm, but the subproblems are not necessarily of the same size.



Tony Hoare (Turing Award 1980)

- Seminar 3 will cover it in detail.

## Other Divide-and-Conquer examples

The Divide-and-Conquer paradigm can be applied to get efficient algorithms for a wide range of problems, such as:

- Finding closest pair of points in a plane in  $O(n \log n)$ .
- Counting inversions in  $O(n \log n)$ , Applied Class next week.
- Improving matrix multiplication (Strassen's algorithm).
- Fast Fourier Transform: this algorithm published by James Cooley and John Tukey in 1965 is one of the most influential algorithms, with a wide range of applications in engineering, music, science, mathematics, etc.
  - ▶ In fact, it can be traced back to unpublished work by Gauss.

# Complexity Analysis

---

# Time complexity

- **Time complexity** is the amount of time taken by an algorithm to run as a function of the input size.
  - ▶ Worst-case complexity (our main focus).
  - ▶ Best-case complexity.
  - ▶ Average-case complexity.



# Asymptotic notation

## Big-O Notation

It is said that  $f(n) = O(g(n))$  if there are constants  $c$  and  $n_0$  such that  $f(n) \leq c \cdot g(n)$  for all  $n \geq n_0$ .

## Big- $\Omega$ Notation

It is said that  $f(n) = \Omega(g(n))$  if there are constants  $c$  and  $n_0$  such that  $f(n) \geq c \cdot g(n)$  for all  $n \geq n_0$ .

## Big- $\Theta$ Notation

It is said that  $f(n) = \Theta(g(n))$  if, and only if,  $f(n) = O(g(n))$  and  $f(n) = \Omega(g(n))$ .

# Space complexity

- **Space complexity** is the total amount of space taken by an algorithm as a function of input size.
- **Auxiliary space complexity** is the amount of space taken by an algorithm *excluding the space taken by the input*.
  - ▶ Many textbooks and online resources do not distinguish between the above two terms and use the term “space complexity” when they are in fact referring to auxiliary space complexity.
  - ▶ In this unit, we use these two terms to differentiate between them.

# In-place algorithm

- An **in-place algorithm** has  $O(1)$  auxiliary space complexity.
  - ▶ In other words, it only requires constant space in addition to the space taken by its input.
  - ▶ Merging is not an in-place algorithm as it needs to create the output list which is size  $n$ .
  - ▶ Be mindful that some books use a different definition (e.g., space taken by recursion may be ignored). For the sake of this unit, we will use the above definition.

# Time complexity of Binary Search

---

## Binary Search

---

```
1: function BINARY_SEARCH(array[1..n], key)
2:   lo = 1 and hi = n + 1
3:   while lo < hi - 1 do
4:     mid =  $\lfloor (\textit{lo} + \textit{hi}) / 2 \rfloor$ 
5:     if key  $\geq$  array[mid] then lo = mid
6:     else hi = mid
7:   if array[lo] = key then return lo
8:   else return null
```

---

- Worst-case time complexity?

- ▶ Search space at start:  $n$
- ▶ After 1st iteration:  $n/2$
- ▶ After 2nd iteration:  $n/4$
- ▶ ...
- ▶ After  $x$  iterations: 1
- ▶ How many iterations?
- ▶  $O(\log n)$

- Best-case time complexity?

- ▶  $O(1)$

# Space complexity of Binary Search

---

## Binary Search

---

```
1: function BINARY_SEARCH(array[1..n], key)
2:   lo = 1 and hi = n + 1
3:   while lo < hi - 1 do
4:     mid =  $\lfloor (\textit{lo} + \textit{hi}) / 2 \rfloor$ 
5:     if key  $\geq$  array[mid] then lo = mid
6:     else hi = mid
7:   if array[lo] = key then return lo
8:   else return null
```

---

- Space complexity?
  - ▶  $O(n)$
- Auxiliary space complexity?
  - ▶  $O(1)$
- It is an in-place algorithm!

# What is the time complexity?

- **Problem:** Given a sorted array of  $n$  unique numbers and two values  $x$  and  $y$ , print all numbers that are in between  $x$  and  $y$ .
- **Algorithm:** Binary search to find the smallest number greater than  $x$ . Linear scan from  $x$  until next number is  $\geq y$ .
- What is the time complexity?
- $O(n)$  because all numbers may be between  $x$  and  $y$ .
- But it seems to really depend on the output ...

## Output-sensitive time complexity

- **Output-sensitive time complexity** is the time-complexity that also depends on the size of the output.

$$x = 23, y = 35$$

|   |   |   |    |    |    |    |    |    |    |
|---|---|---|----|----|----|----|----|----|----|
| 1 | 5 | 8 | 17 | 22 | 27 | 31 | 32 | 36 | 41 |
|---|---|---|----|----|----|----|----|----|----|

- **Algorithm:** Binary search to find the smallest number greater than  $x$ . Linear scan from  $x$  until next number is  $\geq y$ .
- Let  $w$  be the number of values in the range (i.e., in output). What is the output-sensitive time complexity of the algorithm?
- $O(w + \log n)$ . Note that  $w$  may be  $n$  in the worst-case.
- Output-sensitive complexity is only relevant when output-size may vary, e.g., it is not relevant for sorting, finding minimum value etc.

# Solving Recurrence Relations

---



- A recurrence relation is an equation that recursively defines a sequence of values, and one or more base cases are given, e.g.:

$$T(n) = \begin{cases} T(n-1) + c, & \text{if } n > 1, \\ b, & \text{if } n = 1. \end{cases}$$

- The complexity of recursive algorithms can be analysed by writing its recurrence relation and then solving it.

# Solving a simple recurrence relation for time complexity

---

Power1

---

```
1: function POWER1(x, n)
2:   if  $n = 0$  then return 1
3:   else if  $n = 1$  then return  $x$ 
4:   else return  $x \cdot \text{POWER1}(x, n - 1)$ 
```

---

## Goal

Reduce general case to be in terms of the base case.

## Solution

$$\begin{aligned} T(n) &= b + c \cdot (n - 1) \\ &= c \cdot n + b - c \\ &= O(n) \end{aligned}$$

- Cost when  $n = 1$ :
  - ▶  $T(1) = b$  for constant  $b$
- Cost for general case:
  - ▶  $T(n) = T(n - 1) + c$  for constant  $c$
  - ▶  $T(n) = T(n - 2) + 2c$   
 $T(n) = T(n - 3) + 3c$
  - ▶ Pattern?
  - ▶  $T(n) = T(n - k) + c \cdot k$
  - ▶ Set  $k = n - 1$  to get base case.

# Checking solution by substitution

---

Power1

---

```
1: function POWER1(x, n)
2:   if  $n = 0$  then return 1
3:   else if  $n = 1$  then return  $x$ 
4:   else return  $x \cdot \text{POWER1}(x, n - 1)$ 
```

---

## Goal

Reduce general case to be in terms of the base case.

## Solution

$$\begin{aligned}T(n) &= b + c \cdot (n - 1) \\&= c \cdot n + b - c \\&= O(n)\end{aligned}$$

- Cost when  $n = 1$ :
  - $T(1) = b$  for constant  $b$

- We have that:

$$\begin{aligned}T(1) &= c \cdot 1 + b - c \\&= b\end{aligned}$$

- Cost for general case:
  - $T(n) = T(n - 1) + c$  for constant  $c$

- We have that:

$$\begin{aligned}T(n - 1) + c &= c \cdot (n - 1) + b - c + c \\&= c \cdot n + b - c \\&= T(n)\end{aligned}$$

# Space complexity of this power function

---

## Power1

---

```
1: function POWER1( $x, n$ )  
2:   if  $n = 0$  then return 1  
3:   else if  $n = 1$  then return  $x$   
4:   else return  $x \cdot \text{POWER1}(x, n - 1)$ 
```

---

---

## Power2

---

```
1: function POWER2( $x, n$ )  
2:    $result = 1$   
3:   for  $i \leftarrow 1$  to  $n$  do  
4:      $result = result \cdot x$   
5:   return  $result$ 
```

---

- Space complexity?
  - ▶ Total space usage = local space used by the function \* maximum depth of recursion
  - ▶  $O(n)$
- We will not discuss tail-recursion in this unit because it is language specific, e.g., Python doesn't utilise tail-recursion.
- Auxiliary Space Complexity?
- $\text{POWER1}(x, n)$  is not in-place.
- Iterative version  $\text{POWER2}(x, n)$  is in-place.

# Yet another power function

---

## Power3

---

```
1: function POWER3(x, n)
2:   if  $n = 0$  then return 1
3:   else if  $n = 1$  then return  $x$ 
4:    $y = \text{POWER3}(x \cdot x, \lfloor n/2 \rfloor)$ 
5:   if  $n$  even then return  $y$ 
6:   else return  $x \cdot y$ 
```

---

### Goal

Reduce general case to be in terms of the base case.

### Solution

$$\begin{aligned} T(n) &= b + c \cdot \log n \\ &= O(\log n) \end{aligned}$$

- Cost when  $n = 1$ :
  - ▶  $T(1) = b$  for constant  $b$
- Cost for general case:
  - ▶  $T(n) = T(n/2) + c$  for constant  $c$
  - ▶  $T(n) = T(n/4) + 2c$   
 $T(n) = T(n/8) + 3c$
  - ▶ Pattern?
  - ▶  $T(n) = T\left(\frac{n}{2^k}\right) + c \cdot k$
  - ▶ Set  $k = \log n$  to get base case.

# Another check by substitution

---

## Power3

---

```
1: function POWER3(x, n)
2:   if  $n = 0$  then return 1
3:   else if  $n = 1$  then return  $x$ 
4:    $y = \text{POWER3}(x \cdot x, \lfloor n/2 \rfloor)$ 
5:   if  $n$  even then return  $y$ 
6:   else return  $x \cdot y$ 
```

---

### Goal

Reduce general case to be in terms of the base case.

### Solution

$$\begin{aligned} T(n) &= b + c \cdot \log n \\ &= O(\log n) \end{aligned}$$

- Cost when  $n = 1$ :
  - $T(1) = b$  for constant  $b$

- We have that:

$$\begin{aligned} T(1) &= b + c \cdot \log 1 \\ &= b \end{aligned}$$

- Cost for general case:
  - $T(n) = T(n/2) + c$  for constant  $c$

- We have that:

$$\begin{aligned} T(n/2) + c &= b + c \cdot \log(n/2) + c \\ &= b + c \cdot (\log n - \log 2) + c \\ &= b + c \cdot \log n \\ &= T(n) \end{aligned}$$

- Recurrence relation:

$$T(n) = \begin{cases} T(n/2) + c, & \text{if } n > 1, \\ b, & \text{if } n = 1. \end{cases}$$

- Algorithmic example?
  - ▶ Binary search
- Asymptotic complexity?
  - ▶  $O(\log n)$

- Recurrence relation:

$$T(n) = \begin{cases} T(n-1) + c, & \text{if } n > 1, \\ b, & \text{if } n = 1. \end{cases}$$

- Algorithmic example?
  - ▶ Linear search
- Asymptotic complexity?
  - ▶  $O(n)$



- Recurrence relation:

$$T(n) = \begin{cases} 2 \cdot T(n/2) + c \cdot n, & \text{if } n > 1, \\ b, & \text{if } n = 1. \end{cases}$$

- Algorithmic example?
  - ▶ Merge Sort
- Asymptotic complexity?
  - ▶  $O(n \log n)$

- Recurrence relation:

$$T(n) = \begin{cases} T(n-1) + c \cdot n, & \text{if } n > 1, \\ b, & \text{if } n = 1. \end{cases}$$

- Algorithmic example?
  - ▶ Selection Sort
- Asymptotic complexity?
  - ▶  $O(n^2)$

- Recurrence relation:

$$T(n) = \begin{cases} 2 \cdot T(n-1) + c, & \text{if } n > 1, \\ b, & \text{if } n \in \{0, 1\}. \end{cases}$$

- Algorithmic example?
  - ▶ Naive recursive Fibonacci
- Asymptotic complexity?
  - ▶  $O(2^n)$

- Course Notes: Sections 1.2, 1.3 and 1.4, Chapter 2
- Additional resources (not required, but also not necessary covering all topics). Contents related to this seminar and recap of previous units can be found in standard algorithms books such as:
  - ▶ CLRS: Chapters 3 and 4
  - ▶ KT: Chapters 2 and 5
  - ▶ Rou: Chapters 1 to 4

# Concluding remarks

- This unit demands your efforts from Week 1.
- The Divide-and-Conquer algorithm design paradigm can be useful for reducing time complexity.
- Coming up next:
  - ▶ Analysis of algorithms
  - ▶ Non-comparison based sorting (Counting Sort, Radix Sort)
- Preparation required before the next week:
  - ▶ Revise computational complexity covered in earlier units.
  - ▶ Complete Preparation 1 (in your own time to self-assess your prerequisite knowledge).
  - ▶ Complete Preparation 2 before your Applied Class of Week 2.